Guide complet — Réplication de Guéant (2017) : Optimal Market Making

---

A) Résumé du papier et du code

Le papier

Guéant (2017) "*Optimal Market Making*" propose un cadre unifié pour le market making optimal dans la lignée d'Avellaneda-Stoikov, mais avec des **fonctions d'intensité générales** (pas uniquement exponentielles). La contribution majeure est triple : (1) montrer que l'équation HJB à 4 variables se réduit à un **système d'EDO ordinaires** via un changement de variable θ, réconciliant ainsi les différents critères d'optimisation de la littérature (Model A = utilité CARA, Model B = mean-variance) ; (2) dériver des **approximations en forme fermée** pour les quotes optimales, généralisant les formules de Guéant-Lehalle-Fernandez-Tapia ; (3) étendre le modèle au cas **multi-actif** (d assets corrélés), avec les premières approximations en forme fermée pour un market maker multi-actif. L'application numérique porte sur deux indices de crédit : CDX.NA.IG et CDX.NA.HY.

Le code

Le repository implémente fidèlement les Sections 3–6 du papier. Il produit :

- **Figures 1–5 (IG) et 10–14 (HY)** : quotes optimales en fonction du temps et de l'inventaire (δ^bid, δ^ask, spread, skew)
- **Figures 2–7** : comparaison ODE exact vs approximations en forme fermée (y compris à σ/2)
- **Figures 8–9 (IG) et 15–16 (HY)** : comparaison Model A vs Model B
- **Figures 17–19** : cas multi-actif 2D (surfaces 3D, effet croisé de la corrélation)
- **Extension Monte Carlo** (hors papier) : histogramme P&L, trajectoires, contrôle d'inventaire

---

B) Cartographie du code

Arborescence reconstituée

Le projet s'organise en 4 couches logiques. Note : certains fichiers existent dans la base de connaissances du projet mais sont **vides sur disque** (`params.py`, `ode_solver_2d.py`, `simulator.py`, `test_intensity.py`). Les imports indiquent une structure parent avec `src/` et `asset/` :

```
PROJECT/
├── asset/                # Couche données
│   ├── params.py             # Paramètres IG, HY, γ, ϱ, T (Section 6)
│   └── simulator.py          # Monte Carlo (extension hors papier)
│
```

```
├── src/                    # Couche calcul (noyau mathématique)
│   ├── intensity.py        # Λ(δ), C(ξΔ), H_ξ(p), δ*(p)
│   ├── closed_form.py      # Approximations Guéant-Lehalle-FT (Section 4)
│   ├── ode_solver_1d.py    # Solveurs 1D : Model A (linéaire) + B/général (Newton)
│   └── ode_solver_2d.py    # Solveur 2D : multi-actif (Section 5)
│
├── 01_single_asset.py      # Figures 1–5, 10–14 (Section 6.1)
├── 02_closed_form.py       # Figures 2–7, 11–14 (Section 4 vs 6)
├── 03_model_a_vs_b.py      # Figures 8–9, 15–16 (Section 6.2)
├── 04_multi_asset.py       # Figures 17–19 (Section 6.3)
├── 05_monte_carlo.py       # Extension : P&L, trajectoires
│
├── 01–05_*.ipynb           # Notebooks Jupyter (wrappers des .py)
├── __init__.py             # Package marker (vide)
├── test_intensity.py       # Tests unitaires pour intensity.py (vide)
└── optimal_market_making_*.pdf   # Le papier de référence
```

Flow complet

```
params.py (σ, A, k, Δ, Q, γ, ϱ, T)
   │
   ▼
intensity.py        ← briques math de base : Λ, C, H, H', H'', δ*
   │
   ├──▶ ode_solver_1d.py
   │      ├── solve_model_a()    ← v-transform linéaire (Eq. 3.13)
   │      ├── solve_model_b()    ← Newton implicite, ξ=0
   │      └── solve_general()    ← Newton implicite, ξ quelconque
   │          ↓
   │      θ(t,n), δ^b(t,n), δ^a(t,n)
   │
   ├──▶ ode_solver_2d.py
   │      └── solve_2d()         ← Newton sparse 2D (Eq. 5.13)
   │          ↓
   │      θ(t,n₁,n₂), δ^b_i, δ^a_i
   │
   ├──▶ closed_form.py
   │      └── approx_quotes()    ← formules (4.6)–(4.7)
   │
   └──▶ simulator.py
          ├── simulate_1d()      ← MC avec quotes ODE
          └── simulate_naive()   ← MC avec spread fixe
              ↓
          P&L, inventaire, trajectoires

Scripts 01–05 : orchestration + matplotlib → figures/fig*.png
```

C) Walkthrough détaillé

C.1 — params.py : les données du problème

Ce que ça fait : Définit les constantes numériques exactes de la Section 6 du papier pour les deux indices de crédit CDX.NA.IG et CDX.NA.HY.

Entrées/sorties : Pas de calcul — export de dictionnaires IG, HY et de scalaires GAMMA, RHO, T.

Paramètres et leurs rôles :

Paramètre	IG	HY	Sens physique
σ	5.83×10^{-6} \$/√s	2.15×10^{-5} \$/√s	Volatilité du taux upfront
A	9.10×10^{-4} /s	1.06×10^{-3} /s	Taux d'arrivée de base des ordres
k	1.79×10^{4} /\$	5.47×10^{3} /\$	Décroissance de l'intensité avec la distance au mid
Δ	50M \$	10M \$	Notionnel par trade (= taille du lot)
Q	4 lots	4 lots	Inventaire max en lots
γ	6×10^{-5} /\$	—	Aversion au risque
ϱ	0.9	—	Corrélation IG-HY
T	7200 s	—	Horizon (2 heures)

Choix silencieux importants :

- L'inventaire est en **lots** ($n \in \{-Q, \dots, +Q\}$), pas en notionnel. Le notionnel $q = n\Delta$.
- σ est la volatilité du **taux upfront** (pas du prix), d'où les ordres de grandeur 10^{-6}.
- Le spread δ est un **montant absolu en \$**, pas un pourcentage.
- ell_func = None partout dans les scripts → **pas de pénalité terminale** ($\ell \equiv 0$), ce qui correspond au cas asymptotique étudié dans le papier.

Erreur fréquente : confondre σ avec une volatilité "classique" de type 20% annualisé. Ici c'est en \$/√s sur le taux upfront.

C.2 — intensity.py : les briques mathématiques

Ce que ça fait : Implémente les formules en forme fermée pour le cas **exponentiel** $\Lambda(\delta) = A\cdot\exp(-k\delta)$, qui est le cas spécial du papier permettant des calculs explicites.

Fonction par fonction :

`Lambda(delta, A, k)` → **Λ(δ)**

- Intensité d'arrivée des ordres à distance δ du mid-price.
- Plus δ est grand (quote agressive = loin du mid), moins on a de fills.
- C'est l'Eq. (2.3) spécialisée au cas exponentiel.

`C_coeff(xi_Delta, k)` → **C(ξΔ)**

- Coefficient qui résulte de l'optimisation du sup dans la définition de H_ξ.
- Formule : `C(ξΔ) = (1 + ξΔ/k)^{−(k/(ξΔ) + 1)}` si $\xi\Delta > 0$, sinon `e⁻¹`.
- Le cas $\xi\Delta = 0$ correspond au Model B (mean-variance) — c'est la limite continue obtenue par L'Hôpital.
- **Hypothèse critique** : la fonction ne gère que des scalaires (pas vectorisée). Le seuil `EPS = 1e-12` sépare les deux régimes.

`H_val(p, xi, A, k, Delta)` → **H_ξ(p)**

- Le **Hamiltonien** de l'EDO réduite. C'est la pièce maîtresse du modèle.
- Formule : `H_ξ(p) = (AΔ/k)·C(ξΔ)·exp(−kp)`
- Dans l'EDO, p = (θ_n − θ_{n±1})/Δ est le "gradient discret" de θ entre inventaires adjacents.
- Le Hamiltonien capture le trade-off : quand tu quotes large (p petit → H grand), tu gagnes peu par fill mais tu as plus de fills.

`H_prime(p, ...)` et `H_second(p, ...)`

- Dérivées analytiques : $H'(p) = -k \cdot H(p)$, $H''(p) = k^2 \cdot H(p)$.
- Utilisées dans le solveur Newton (Jacobien) et dans la closed-form (ω dépend de H''(0)).

`delta_star(p, xi, k, Delta)` → **δ*(p)**

- Le **spread optimal** qui réalise le sup dans la définition de H_ξ.
- Formule : `δ*(p) = p + (1/(ξΔ))·ln(1 + ξΔ/k)` si $\xi\Delta > 0$, sinon `p + 1/k`.
- C'est l'Eq. (3.14)/(3.16) du papier. On l'applique après avoir résolu l'EDO pour extraire les quotes.
- **Vectorisée en p** (appels sur des tableaux NumPy entiers).

Pourquoi c'est nécessaire : Toutes les autres couches dépendent de ces fonctions. Elles encodent la micro-structure du marché (comment l'intensité de fill dépend de la distance au mid).

C.3 — `ode_solver_1d.py` : le cœur du solveur

Ce fichier implémente la résolution numérique de l'EDO (3.9) du papier, spécialisée au cas 1D (un seul actif). C'est le fichier le plus long et le plus critique.

L'EDO à résoudre

$$\partial_t \theta_n + \tfrac{1}{2}\gamma\sigma^2(n\Delta)^2 - \mathbb{1}_{n<Q}\, H_\xi((\theta_n - \theta_{n+1})/\Delta) - \mathbb{1}_{n>-Q}\, H_\xi((\theta_n - \theta_{n-1})/\Delta) = 0$$

- $\theta_n(t)$ est la **fonction valeur réduite** au lot n et temps t.
- Le terme $\tfrac{1}{2}\gamma\sigma^2(n\Delta)^2$ est la **pénalité d'inventaire** (risque de prix).
- Les termes $H_\xi(...)$ capturent le **bénéfice attendu** des fills bid/ask.
- Condition terminale : $\theta_n(T) = -\ell(|n\Delta|)$, avec $\ell \equiv 0$ dans les scripts.

`solve_model_a(params, gamma, T, N_t, ell_func)` — Solveur linéaire

Idée clé (Eq. 3.13) : Pour $\xi = \gamma$ (Model A) avec intensités exponentielles, le changement de variable `v_n = exp(−k·θ_n/Δ)` transforme l'EDO non-linéaire en système **linéaire** :

$$\partial_\tau v_n = -\tfrac{1}{2}k\gamma\sigma^2 n^2\Delta \cdot v_n + AC_{\xi\Delta} \cdot (v_{n+1} + v_{n-1})$$

C'est un **système tridiagonal constant** (les coefficients ne dépendent pas de v) !

Schéma numérique : Euler implicite en temps rétrograde $\tau = T - t$:

$$(I + dt \cdot M) \cdot v^{m+1} = v^m$$

où M est la matrice tridiagonale avec :

- Diagonale : `α·n²` avec $\alpha = \tfrac{1}{2}k\gamma\sigma^2\Delta$
- Hors-diagonale : `−β` avec $\beta = AC_{\xi\Delta}$

Choix silencieux :

- `N_t = 7200` (1 pas/seconde pour T = 7200s) — choix raisonnable mais conservateur
- `np.maximum(v, 1e-300)` : garde numérique pour éviter log(0) lors de la conversion $v \to \theta$
- La matrice banded est **constante** → on la factorise une fois (implicitement via `solve_banded`)
- Convention banded de scipy : ab[0] = super-diag, ab[1] = diag, ab[2] = sub-diag

Erreur fréquente : Le signe de la v-transform. Le papier utilise `v = exp(−kθ/Δ)`, le code `v = exp(+kθ/Δ)` (visible ligne 88 : $\theta = +(\Delta/k)\cdot\ln(v)$). En fait les deux sont cohérents car θ dans le code correspond à −θ dans certaines conventions du papier. Le code est correct.

`solve_model_b(params, gamma, T, N_t, ell_func)` — Newton implicite

Pour $\xi = 0$ (Model B), la v-transform ne linéarise pas. On utilise Newton sur chaque pas d'Euler implicite.

Pour $\xi = 0$ (Model B), la v-transform ne linéarise pas. On utilise Newton sur chaque pas d'Euler implicite.

Schéma : À chaque pas de temps m, on cherche θ^{new} tel que :

```
G(θ^{new}) = θ^{new} − θ^{old} + dt·F(θ^{new}) = 0
```

Newton : $\theta \leftarrow \theta - J^{-1} \cdot G$, où J est le Jacobien tridiagonal de G.

Choix silencieux :

- 12 itérations Newton max — suffisant car la convergence quadratique fait que 3–5 itérations suffisent typiquement
- Convergence : `max|corr| < 1e-15` — critère très strict
- Guess initial : θ^{old} (solution du pas précédent) — excellent car les pas sont petits

`solve_general(params, gamma, T, xi, N_t, ell_func)` — Solveur universel

Identique à `solve_model_b` mais avec ξ arbitraire. **C'est le solveur utilisé dans tous les scripts.** Le Model A linéaire (`solve_model_a`) est disponible mais pas appelé — `solve_general` avec $\xi = \gamma$ donne les mêmes résultats via Newton.

Pourquoi garder les deux ? Le solveur linéaire est ~10x plus rapide (pas de Newton) et sert de **validation croisée**.

`_newton_residual(...)` — Résidu et Jacobien

C'est la fonction interne la plus dense. Pour chaque lot n :

1. Calcule la pénalité d'inventaire : `½γσ²(nΔ)²`
2. Calcule le gradient discret p_bid = (θ_n − θ_{n+1})/Δ et p_ask = (θ_n − θ_{n−1})/Δ
3. Évalue H(p) et H'(p) = −kH(p)
4. Assemble le résidu G_n et les entrées du Jacobien tridiagonal

Subtilité : Les conditions aux bords sont encodées via `𝟙_{n<Q}` et `𝟙_{n>−Q}`. Au lot n = +Q, pas de bid (on est au max d'inventaire). Au lot n = −Q, pas d'ask.

`_extract_quotes(theta, lots, xi, k, Delta, Q)` — De θ aux quotes

Après résolution, on récupère les quotes optimales :

```
p_bid = (θ_n − θ_{n+1})/Δ  →  δ^b_n = δ*(p_bid)
p_ask = (θ_n − θ_{n−1})/Δ  →  δ^a_n = δ*(p_ask)
```

Les cases impossibles (bid au lot +Q, ask au lot −Q) restent à `NaN`.

C.4 — `closed_form.py` : approximations analytiques

C.4 — `closed_form.py` : approximations analytiques

Implémente les Eqs. (4.6)–(4.7) du papier.

`approx_quotes(n, params, gamma, xi)` → (δ^b, δ^a)

Formules :

```
δ^b(n) = δ_static + ω·(2n+1)Δ/2
δ^a(n) = δ_static − ω·(2n−1)Δ/2
```

où :

- `δ_static = (1/(ξΔ))·ln(1 + ξΔ/k)` : composante statique (indépendante de n et t)
- `ω = √(γσ² / (2·A·Δ·k·C(ξΔ)))` : pente dynamique

Pourquoi c'est une approximation : Ces formules supposent qu'on est dans le **régime asymptotique** (loin de T), et résultent d'un développement spectral de l'EDP linéarisée (Section 4.1). Elles sont exactes quand $T \to \infty$ et quand la non-linéarité est faible.

Ce qu'on en déduit :

- Le spread = $\delta^b + \delta^a = 2\cdot\delta_{static} + \omega\cdot\Delta$ est **constant en n** (pour intensités exponentielles)
- Le skew = $\delta^b - \delta^a = 2n\cdot\omega\cdot\Delta$ est **linéaire en n**

C.5 — `ode_solver_2d.py` : le cas multi-actif

Résout l'Eq. (5.13) pour d = 2 assets (IG + HY) avec corrélation ϱ.

Différences avec le 1D :

- La grille est 2D : $(n_1, n_2) \in \{-Q_1,\dots,+Q_1\} \times \{-Q_2,\dots,+Q_2\} \to 81$ points pour Q=4
- La pénalité d'inventaire est **quadratique vectorielle** : $\frac{1}{2}\gamma\cdot q^T\cdot\Sigma\cdot q$ avec la matrice de covariance complète (incluant le terme croisé $\varrho\sigma_1\sigma_2$)
- Le Jacobien est **sparse** (pas tridiagonal) : chaque nœud (n_1,n_2) est couplé à ses 4 voisins
- On utilise `scipy.sparse.lil_matrix` pour le remplissage, converti en `csc_matrix` pour la résolution

`_build_grid(Q1, Q2)` → grid, idx, N

Construit le dictionnaire de correspondance $(n_1, n_2) \leftrightarrow$ indice flat. Itère n_1 en boucle externe, n_2 en interne.

`_residual_and_jacobian(...)` — Résidu 2D

Même logique que le 1D mais pour 4 directions (bid/ask × 2 assets). Le Jacobien est une matrice creuse N×N avec au plus 5 non-zéros par ligne.

Choix silencieux :

- `N_t = 720` dans le script 04 (vs 7200 en 1D) — compromis vitesse/précision, car 81 points × 720 pas × Newton ≈ quelques minutes
- Convergence Newton : `1e-14` (légèrement plus lâche que le 1D)

C.6 — `simulator.py` : Monte Carlo

`simulate_1d(sol, params, gamma, T, N_sim, seed)` → dict

Logique :

1. Pré-calcule les tables de quotes δ^b[t, n] et δ^a[t, n] depuis la solution ODE
2. Pour chaque trajectoire, simule :
   - Prix mid : $S_{t+1} = S_t + \sigma\cdot\sqrt{dt}\cdot Z$ (marche aléatoire gaussienne)
   - Fills Poisson : $P(fill_bid) = A\cdot\exp(-k\cdot\delta^b)\cdot dt$, test via uniforme $U < p$
   - Cash : $X \leftarrow X - (S - \delta^b)\cdot\Delta$ sur bid fill, $X \leftarrow X + (S + \delta^a)\cdot\Delta$ sur ask fill
   - Inventaire : $n \leftarrow n \pm 1$ par fill
3. P&L terminal = $X_T + n_T\cdot\Delta\cdot S_T$

Choix silencieux :

- `seed = 42` : reproductibilité
- $S_0 = 0$: on travaille en prix relatif (le mid absolu n'importe pas)
- L'approximation Poisson via $P(fill) = \lambda\cdot dt$ est correcte quand dt est petit (dt = 1s ici)
- Les fills bid et ask utilisent des tirages U **indépendants** (pas corrélés)

`simulate_naive(...)` → dict

Même structure mais avec un spread fixe (celui du lot n=0). Sert de benchmark pour montrer la valeur de la stratégie optimale.

C.7 — Scripts 01–05 : orchestration

Script	Figures reproduites	Quoi
`01_single_asset.py`	1–5 (IG), 10–14 (HY)	δ^b(t,n), δ^b(0,n), δ^a(0,n), spread, skew
`02_closed_form.py`	2–7 (IG σ et σ/2), 11–14 (HY)	ODE exact vs closed-form overlay
`03_model_a_vs_b.py`	8–9 (IG), 15–16 (HY)	Model A (ξ=γ) vs Model B (ξ=0)

04_multi_asset.py	17–19	Surfaces 3D + cross-section ϱ
05_monte_carlo.py	A, B, C (extension)	P&L, trajectoire, stats inventaire

D) Mapping exact avec le papier

D.1 — Table de correspondance

Papier	Équation / Figure	Code	Fichier : fonction
Intensité Λ(δ) = Ae^{−kδ}	Eq. (2.3), cas exp.	Lambda()	intensity.py:23
Hamiltonien H_ξ(p)	Après Eq. (3.9)	H_val()	intensity.py:38
Coefficient C_ξ	Après Eq. (3.13)	C_coeff()	intensity.py:29
Spread optimal δ*(p)	Eq. (3.14)/(3.16)	delta_star()	intensity.py:55
EDO réduite sur θ	Eq. (3.9)	_newton_residual()	ode_solver_1d.py:207
v-transform linéaire	Eq. (3.13)	solve_model_a()	ode_solver_1d.py:29
Approximation fermée bid	Eq. (4.6)	approx_quotes()	closed_form.py:29
Approximation fermée ask	Eq. (4.7)	approx_quotes()	closed_form.py:29
Spread ≈ 2δ_static + ωΔ	Eq. (4.8)	approx_spread()	closed_form.py:67
Skew ≈ 2nωΔ	Eq. (4.9)	approx_skew()	closed_form.py:72
EDO multi-actif	Eq. (5.13)	solve_2d()	ode_solver_2d.py
Matrice Σ	Eq. (5.1)	Sig dans solve_2d()	ode_solver_2d.py
Paramètres IG/HY	Section 6, Table implicite	IG, HY	params.py
Model A vs B	ξ=γ vs ξ=0	xi parameter	Partout
Fig 1 : δ^b(t,n) IG	Section 6	plot_asset()	01_single_asset.py:36
Fig 8–9 : A vs B	Section 6	compare_ab()	03_model_a_vs_b.py:41
Fig 17–18 : surfaces 3D	Section 6	surface_plot()	04_multi_asset.py:42
Fig 19 : cross-section ϱ	Section 6	Boucle rho_values	04_multi_asset.py:80

D.2 — Fidélité de la réplication

Répliqué fidèlement :

- ✅ Le cadre Avellaneda-Stoikov avec intensités exponentielles
- ✅ La réduction HJB → EDO via l'ansatz θ (Eqs. 3.3, 3.7)
- ✅ La v-transform linéaire pour Model A (Eq. 3.13)
- ✅ Le solveur Newton implicite pour Model B et le cas général
- ✅ Les approximations en forme fermée (Eqs. 4.6–4.9)
- ✅ Le cas multi-actif 2D avec corrélation (Eq. 5.13)
- ✅ Les paramètres numériques exacts de la Section 6
- ✅ Toutes les 19 figures principales

Écarts identifiés :

- ⚠️ Le script 01 utilise `solve_general` (Newton) au lieu de `solve_model_a` (linéaire) pour Model A → même résultat mais plus lent
- ⚠️ `N_t = 720` pour le 2D (au lieu de 7200) → léger manque de précision temporelle
- ⚠️ Pas de pénalité terminale ($\ell \equiv 0$) dans les scripts — le papier mentionne cette possibilité mais ne l'utilise pas dans les figures
- ⚠️ Le papier montre des figures en termes de q (notionnel), le code utilise n (lots) en abscisse — même information, présentation différente

Non implémenté :

- ❌ Intensités générales (non exponentielles) — Sections 3–4 en toute généralité
- ❌ Approximations multi-actif en forme fermée (Section 5, après Eq. 5.13)
- ❌ Cas $d > 2$ assets
- ❌ Vérification argument formel (Theorem 3.2) — c'est une preuve mathématique
- ❌ Pénalité terminale non nulle $\ell(|q|)$

E) Validation / Reproduction

E.1 — Checklist de reproduction

Environnement :

```bash
```

```
Python 3.9+
numpy >= 1.21
scipy >= 1.7
matplotlib >= 3.4
```

Commandes :

```bash
python 01_single_asset.py    # ~30s  → 10 figures
python 02_closed_form.py     # ~45s  → 3 figures
python 03_model_a_vs_b.py    # ~60s  → 2 figures
python 04_multi_asset.py     # ~10min → 3 figures (4 solves 2D)
python 05_monte_carlo.py     # ~2min  → 3 figures
```

⚠️ **Problème actuel** : Les imports `from asset.params import ...` et `from src.ode_solver_1d import ...` supposent une structure de répertoires `asset/` et `src/` dans un répertoire parent, avec les fichiers correctement placés. Les fichiers `params.py`, `ode_solver_2d.py`, `simulator.py` et `test_intensity.py` sont **vides sur disque** — ils doivent être remplis avec le contenu présent dans la base de connaissances du projet.

E.2 — Sanity checks

1. **C(ξΔ) au limites** : `C_coeff(0, k)` doit retourner $e^{-1} \approx 0.3679$. Pour $\xi\Delta \to \infty$, $C \to 0$.
2. *δ(0)** : `delta_star(0, gamma, k, Delta)` doit retourner $(1/(\gamma\Delta))\cdot\ln(1 + \gamma\Delta/k)$. Vérifier numériquement :
   - IG : $\xi\Delta = 6e\text{-}5 \times 50e6 = 3000$, $\delta^*(0) = (1/3000)\cdot\ln(1 + 3000/17900) \approx 5.43e\text{-}5$
   - HY : $\xi\Delta = 6e\text{-}5 \times 10e6 = 600$, $\delta^*(0) = (1/600)\cdot\ln(1 + 600/5470) \approx 1.80e\text{-}4$
3. **Symétrie** : θ_n(0) doit être symétrique en n (θ_n = θ_{−n}), donc δ^b(0,n) = δ^a(0,−n).
4. **Régime asymptotique** : Les quotes doivent se stabiliser bien avant T = 7200s. Si on voit que δ^b(t,n) n'a pas convergé à t = 0, augmenter T.
5. **Model A ≈ Model B** : Le max|δ^b_A − δ^b_B| doit être petit (< 1% relatif).
6. **Spread constant** (closed-form) : Pour intensités exponentielles, le spread δ^b + δ^a est indépendant de n dans l'approximation. L'ODE exact doit s'en approcher.

E.3 — Résultats attendus (ordres de grandeur)

Métrique	IG (Model A)	HY (Model A)
δ^b(0, n=0)	~5–6 × 10^{-5} $	~2 × 10^{-4} $
Spread(0, n=0)	~1.1 × 10^{-4} $	~3.6 × 10^{-4} $
Skew(0, n=0)	0 (par symétrie)	0

Convergence asymptotique	< 1h	< 1h
Max	A – B	bid (relatif)

E.4 — Diagnostic de divergence

- **θ explose** → dt trop grand (réduire) ou erreur de signe dans l'EDO
- **Newton ne converge pas** → vérifier le Jacobien (signes des dérivées de H)
- **v_n < 0 dans le solveur linéaire** → instabilité numérique (la garde `1e-300` devrait empêcher)
- **Spread négatif** → erreur dans les conventions bid/ask ou dans p = (θ_n − θ_{n+1})/Δ
- **Quotes non symétriques** → bug dans _extract_quotes ou conditions aux bords

---

F) Roadmap "reste à faire"

Priorité 1 — Indispensable pour "on a répliqué"

#	Tâche	Effort	Risque	Dépendances
F1	**Remplir les fichiers vides** (params.py, ode_solver_2d.py, simulator.py, test_intensity.py) avec le contenu de la knowledge base	S	Bas	Aucune
F2	**Fixer la structure d'import** (créer les dossiers src/ et asset/, ou adapter les imports)	S	Bas	F1
F3	**Écrire les tests unitaires** pour intensity.py (C aux limites, δ* symétrie, H dérivées)	S	Bas	F2
F4	**Valider les figures vs le papier** : superposer numériquement les outputs aux figures PDF	M	Moyen	F2
F5	**Ajouter N_t = 7200 au solveur 2D** et vérifier convergence	M	Moyen (temps de calcul)	F2

Priorité 2 — Bonus (solidifie la réplication)

#	Tâche	Effort	Risque
F6	Implémenter `solve_model_a` comme fast-path dans les scripts et comparer au Newton	S	Bas
F7	Ajouter la pénalité terminale ℓ(	q	) non nulle (ex: ℓ = α

F8	Tests de convergence : varier N_t et montrer que les quotes convergent	M	Bas
F9	Profiling et optimisation du solveur 2D (vectoriser le Jacobien, pré-allouer la sparse)	M	Moyen
F10	Ajouter le cas σ/2 pour HY (seulement fait pour IG actuellement)	S	Bas

Priorité 3 — Paper-complete

#	Tâche	Effort	Risque
F11	Intensités générales (power-law, etc.) avec inversion numérique de Λ^{-1}	L	Élevé
F12	Approximations multi-actif en forme fermée (Section 5.3)	M	Moyen
F13	Cas d > 2 (3+ assets) — explosion combinatoire de la grille	L	Élevé

G) Extensions CV-impactantes

Quick wins (1–2 jours)

G1. Analyse de sensibilité γ : Varier γ ∈ {1e-5, 3e-5, 6e-5, 1e-4, 3e-4} et tracer spread(γ), skew(γ), fill rate(γ). Montre que tu comprends le rôle de l'aversion au risque. → Métriques : spread moyen, pente du skew, nombre de fills/heure.

G2. Efficient frontier P&L-risk : Utiliser le MC (script 05) pour tracer E[P&L] vs Std[P&L] pour différents γ. C'est la "frontière efficiente du market maker". → Métriques : Sharpe ratio, VaR 5%, max drawdown.

G3. Comparaison closed-form vs ODE en temps réel : Mesurer le temps CPU des deux approches. Montrer que closed-form est 1000x plus rapide et quantifier l'erreur résiduelle. → Métriques : temps CPU, max|erreur| en δ, erreur relative spread.

Projets solides (1–2 semaines)

G4. Adverse selection / alpha signal : Ajouter un drift μ au prix mid (Eq. 2.1 : dS = μdt + σdW). Le market maker qui anticipe la direction peut skewer ses quotes. Résoudre la nouvelle EDO. → Métriques : P&L moyen, hit ratio directionnel, comparaison vs sans alpha.

G5. Market making sur données réelles (LOB) : Calibrer (σ, A, k) sur données de carnet d'ordres réel (crypto ou actions). Backtest de la stratégie optimale. → Métriques : P&L cumulé, Sharpe, drawdown, comparaison vs benchmarks.

G6. Reinforcement Learning vs ODE analytique : Entraîner un agent RL (DQN/PPO) sur le même

environnement et comparer aux quotes ODE. Montre la valeur de la solution analytique. → Métriques : regret cumulé RL vs optimal, convergence de l'agent, sample efficiency.

G7. Inventory-based spread sur 3+ assets corrélés : Étendre le solveur 2D à d=3 (ex: IG + HY + un 3ème indice). Analyser comment la corrélation croisée affecte les quotes. → Métriques : temps de calcul O(N^d), impact marginal du 3ème actif sur les quotes des 2 premiers.

Ambitieux (1 mois+)

G8. Market making avec impact de prix : Ajouter un impact permanent/temporaire sur le prix (comme Cartea-Jaimungal). L'EDO se complexifie significativement. → Métriques : coût d'impact vs gain de spread, inventaire optimal, comparaison avec/sans impact.

G9. Online learning des paramètres (A, k) : Les paramètres d'intensité dérivent en temps réel. Implémenter un filtre de Kalman ou un estimateur bayésien en ligne qui met à jour (A, k) et recalcule les quotes optimales en continu. → Métriques : tracking error, P&L hors-sample, robustesse au changement de régime.

G10. Production-grade market making engine : Packager le tout en service temps réel : WebSocket pour les prix, calcul de quotes en < 1ms (via closed-form), gestion du risque, logging, dashboard. Déployer sur un testnet crypto. → Métriques : latence, uptime, P&L simulé sur 1 semaine, nombre de fills.

Annexe : Choix silencieux récapitulatifs

Choix	Valeur	Justification
N_t (1D)	7200	1 pas/s, conservateur
N_t (2D)	720	Compromis vitesse, 10x moins que 1D
Newton max iter	12	Convergence quadratique, 3–5 suffisent
Newton tol (1D)	1e-15	Très strict, machine precision
Newton tol (2D)	1e-14	Légèrement relâché
v guard (Model A)	1e-300	Évite log(0)
$\xi\Delta$ seuil	1e-12	Sépare Model A et B numériquement
ell_func	None	Pas de pénalité terminale
S_0 (MC)	0	Prix relatif
seed (MC)	42	Reproductibilité
N_sim	2000	Compromis histogramme/vitesse
Poisson approx	$\lambda \cdot dt$	Valide car dt = 1s et $\lambda \sim 10^{-3}$